

Tecnicatura Universitaria en Programación a Distancia

Trabajo Práctico Integrador - Programación II

Juan José Díaz

Junio 2026

Índice

1. Marco Teórico	2
1.1. Java	2
1.2. Programación Orientada a Objetos (POO)	2
1.3. Colecciones en Java	2
1.4. Enumeraciones	2
1.5. Manejo de Excepciones	2
2. Decisiones Técnicas y Arquitectura	3
2.1. Estructura de Paquetes	3
2.2. Diagrama de Clases UML	4
2.3. Decisiones de Diseño	4
2.4. Capturas de Pantalla	6
3. Dificultades y Soluciones	8
3.1. 1. Alineación con el UML del Profesor	8
3.2. 2. Método <code>calcularTotal()</code> con retorno <code>void</code>	8
3.3. 3. Gestión de IDs en <code>DetallePedido</code>	8
3.4. 4. Limpieza de Pantalla Multiplataforma	8
4. Bibliografía / Webgrafía	9
5. Links	9

1. Marco Teórico

1.1. Java

Java es un lenguaje de programación orientado a objetos, multiplataforma y ampliamente utilizado en el desarrollo de aplicaciones de escritorio, web y móviles. Su máquina virtual (JVM) permite ejecutar el mismo código en diferentes sistemas operativos sin modificaciones. Para este proyecto se utilizó Java 21, la versión LTS más reciente, que incorpora mejoras en switch expressions, patrones de registro y colecciones.

1.2. Programación Orientada a Objetos (POO)

El proyecto sigue los cuatro pilares fundamentales de POO:

- **Herencia:** Todas las entidades heredan de la clase abstracta **Base**, que provee los atributos comunes **id**, **eliminado** y **createdAt**.
- **Encapsulamiento:** Todos los atributos son privados y se accede mediante getters y setters públicos, con validaciones de negocio en los setters (precio y stock no negativos, cantidad mayor a cero).
- **Polimorfismo:** La interfaz **Calculable** es implementada por **Pedido**, permitiendo tratar distintos tipos calculables de forma uniforme.
- **Abstracción:** La clase **Base** define el comportamiento común a todas las entidades, mientras que cada subclase implementa su propia lógica específica.

1.3. Colecciones en Java

El almacenamiento en memoria se realiza mediante la interfaz **List** con implementación **ArrayList**. Esto permite operaciones de agregado, búsqueda y eliminación con soft-delete sobre colecciones dinámicas. Los servicios encapsulan estas colecciones y exponen operaciones CRUD con filtros de eliminados.

1.4. Enumeraciones

Se utilizaron tres enums para representar conjuntos fijos de valores:

- **Rol:** ADMIN, USUARIO
- **Estado:** PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO
- **FormaPago:** TARJETA, TRANSFERENCIA, EFECTIVO

1.5. Manejo de Excepciones

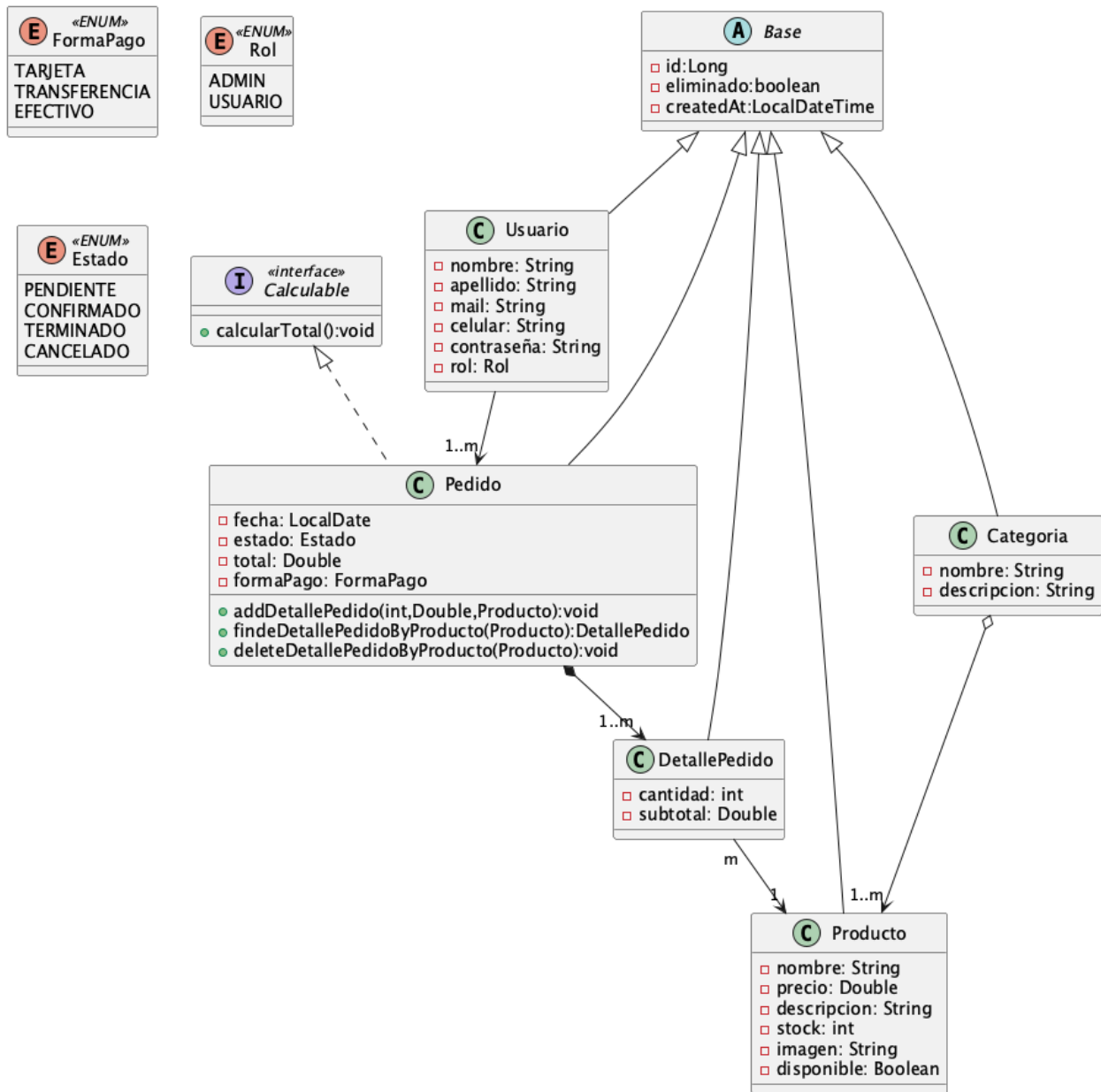
Se implementó una excepción personalizada **FoodStoreException** (extends **RuntimeException**) para validaciones de negocio: precio/stock negativo, cantidad inválida, stock insuficiente, email duplicado, y pedido sin usuario.

2. Decisiones Técnicas y Arquitectura

2.1. Estructura de Paquetes

```
src/integrado/prog2/
+-- Main.java                # Punto de entrada y menu interactivo
+-- config/AppConfig.java    # Constantes de configuracion
+-- entities/                # Clases del modelo UML
|   +-- Base.java            # Abstracta (id, eliminado, createdAt)
|   +-- Categoria.java
|   +-- Producto.java
|   +-- Usuario.java
|   +-- Pedido.java
|   +-- DetallePedido.java
+-- enums/                   # Enumeraciones
|   +-- Rol.java
|   +-- Estado.java
|   +-- FormaPago.java
+-- interfaces/
|   +-- Calculable.java      # Interfaz funcional
+-- exception/
|   +-- FoodStoreException.java # Excepcion de negocio
+-- services/                # Logica de negocio CRUD
    +-- CategoriaService.java
    +-- ProductoService.java
    +-- UsuarioService.java
    +-- PedidoService.java
```

2.2. Diagrama de Clases UML



2.3. Decisiones de Diseño

2.3.1. Soft Delete

En lugar de eliminar físicamente los registros de las colecciones, se marca el atributo `eliminado = true`. Esto preserva la integridad referencial y permite recuperación de datos. Todos los servicios filtran por `!eliminado` en las operaciones de listado y búsqueda.

2.3.2. Composición vs Agregación

La relación entre `Pedido` y `DetallePedido` es de composición (rombo lleno en UML): los detalles pertenecen exclusivamente a un pedido y no tienen existencia independiente. La relación entre `Categoria` y `Producto` es de agregación (rombo vacío): una categoría agrupa productos, pero un producto puede existir sin categoría.

2.3.3. Validaciones en Setters

Las validaciones de negocio se implementan directamente en los setters de las entidades para garantizar que el objeto nunca quede en estado inconsistente, siguiendo el principio de auto-protección del dominio.

2.4. Capturas de Pantalla

2.4.1. Menú Principal

```
===== FOOD STORE =====
1. Gestionar Categorías
2. Gestionar Productos
3. Gestionar Usuarios
4. Gestionar Pedidos
0. Salir
Seleccione una opción:
```

2.4.2. Gestión de Categorías

```
----- CATEGORIAS -----
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
1. Comida | Platos preparados
2. Bebida | Bebidas frias y calientes
3. Postre | Dulces y postres
```

2.4.3. Gestión de Productos

```
----- PRODUCTOS -----
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
1. Hamburguesa | $4500.0 | Stock: 10 | Disp: si | Comida
2. Pizza | $3500.0 | Stock: 8 | Disp: si | Comida
3. Coca Cola | $1200.0 | Stock: 20 | Disp: si | Bebida
4. Agua Mineral | $800.0 | Stock: 15 | Disp: si | Bebida
5. Helado | $2000.0 | Stock: 12 | Disp: si | Postre
6. Flan | $1500.0 | Stock: 10 | Disp: si | Postre
```

2.4.4. Gestión de Usuarios

```
----- USUARIOS -----
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
1. Admin | admin@foodstore.com | Cel: 123456789 | ADMIN
2. Juan Perez | juan@mail.com | Cel: 987654321 | USUARIO
```

2.4.5. Gestión de Pedidos

```
----- PEDIDOS -----
```

1. Listar
2. Crear
3. Actualizar estado
4. Eliminar
5. Listar por usuario
0. Volver

Seleccione: 1

Pedido #1 | Juan Perez | Fecha: 2026-06-25 | Estado: PENDIENTE |

Total: \$10200.0 | Pago: TARJETA

- 2x Hamburguesa (\$9000.0)
- 1x Coca Cola (\$1200.0)

3. Dificultades y Soluciones

3.1. 1. Alineación con el UML del Profesor

Dificultad: Inicialmente se implementó el modelo con atributos como `email/password` en `Usuario`, pero el UML oficial especificaba `mail/contraseña` y atributos adicionales como `apellido`, `celular`, `imagen` y `disponible` que no estaban contemplados.

Solución: Se refactorizaron todas las entidades, servicios y menús para alinearse exactamente con el UML provisto, incluyendo el tipo intencional `findeDetallePedidoByProducto` en `Pedido`.

3.2. 2. Método `calcularTotal()` con retorno `void`

Dificultad: El UML especifica `calcularTotal(): void`, a diferencia del enfoque más común de retornar `double`. Esto implicó que el método debía mutar el estado interno del objeto en lugar de devolver un valor.

Solución: La interfaz `Calculable` se definió con `void calcularTotal()`. El método actualiza el atributo `total` del `Pedido` sumando los subtotales de sus detalles.

3.3. 3. Gestión de IDs en `DetallePedido`

Dificultad: El método `addDetallePedido(int, Double, Producto)` crea el `DetallePedido` internamente, pero no asigna IDs. Los IDs de detalle se necesitan para el reporte.

Solución: En el servicio `PedidoService`, los IDs se asignan manualmente a cada `DetallePedido` antes de agregarlo a la lista del pedido, utilizando el contador `nextId` del servicio.

3.4. 4. Limpieza de Pantalla Multiplataforma

Dificultad: El comando para limpiar la consola difiere entre Windows (`cls`) y sistemas Unix (`clear`), y los códigos ANSI no funcionan en la consola clásica de Windows.

Solución: Se implementó `limpiarPantalla()` que detecta el sistema operativo mediante `System.getProperty("os.name")` y ejecuta el comando apropiado, con un fallback de 50 líneas en blanco si falla.

4. Bibliografía / Webgrafía

- Documentación oficial de Java 21: <https://docs.oracle.com/en/java/javase/21/>
- Oracle Java Tutorials - Collections: <https://docs.oracle.com/javase/tutorial/collections/>
- PlantUML - Diagramas de clases: <https://plantuml.com/class-diagram>
- NetBeans IDE: <https://netbeans.apache.org/>
- Git y GitHub: <https://git-scm.com/>

5. Links

- **Repositorio del proyecto:** <https://github.com/diazjuanjo/UTN-TUPaD-P2-TPI>
- **Video explicativo:** <https://youtu.be/unHsFhyLIv0>